Ankara Üniversitesi ML 2026 WEEK3 Proje Raporu

Hazırlayan:

Civan Nasanlı 24290096

Dersin Öğretim Görevlileri:

Doçent Doktor Mustafa Coşkun

Araştırma Görevlisi Ayşe Elif Yoldaş

Özdeğerler, Özvektörler, Matris Manipülasyonu ve ML ilişkisi

Makine öğrenmesi için matrisler, sadece verilerinin tutulduğu tablolardan daha fazlasıdır.

Bu matrisler verileri temsil eden uzayları şekillendirmemizi sağlar.

ML için kullanılan matrislerin boyutlarını azaltmamız, şekillendirmemiz hangi verinin önemli olduğunu belirlemememiz yani matrisi istediğimiz şekilde manipüle edebilmek için özdeğerleri ve özvektörleri kullanırız.

Bu raporda ise bunu nasıl yaptığımızı, sonuçlarının nasıl alanlarda kullanıldığını ve ne denli önemli olduğunu anlatacağım.

Temel Mantık: Matris Dönüşümleri

Vektörleri Matrisler ile çarparak dönüştürürüz. Bu dönüşümler genel itibariyle vektörlerin yönüyle boyutuyla oynar, uzaydaki şeklini değiştirir.

Ancak, bazı çok özel vektörler vardır ki, bu dönüşümden sonra yönleri hiç değişmez, sadece boyları uzar, kısalır veya yönleri tam tersine (-1 ile çarpılmış gibi) döner.

İşte bu özel vektörlere özvektör (eigenvector), boylarındaki bu değişim miktarına (kat sayısına) ise özdeğer (eigenvalue) denir.

Matematiksel Gösterimi:

$$Av=\lambda v$$

A: Dönüşümü yapan Kare Matris v: Özvektör (Yönü değişmeyen vektör)

λ: Özdeğer (Skaler bir sayıdır. Vektörün ne kadar uzayıp kısaldığını

belirtir)

Bu denklem, A matrisinin v vektörünü yönünü değiştirmeden yalnızca λ katsayısı ile ölçeklediğini ifade eder.

ML'de Nerelerde Kullanılır?

***

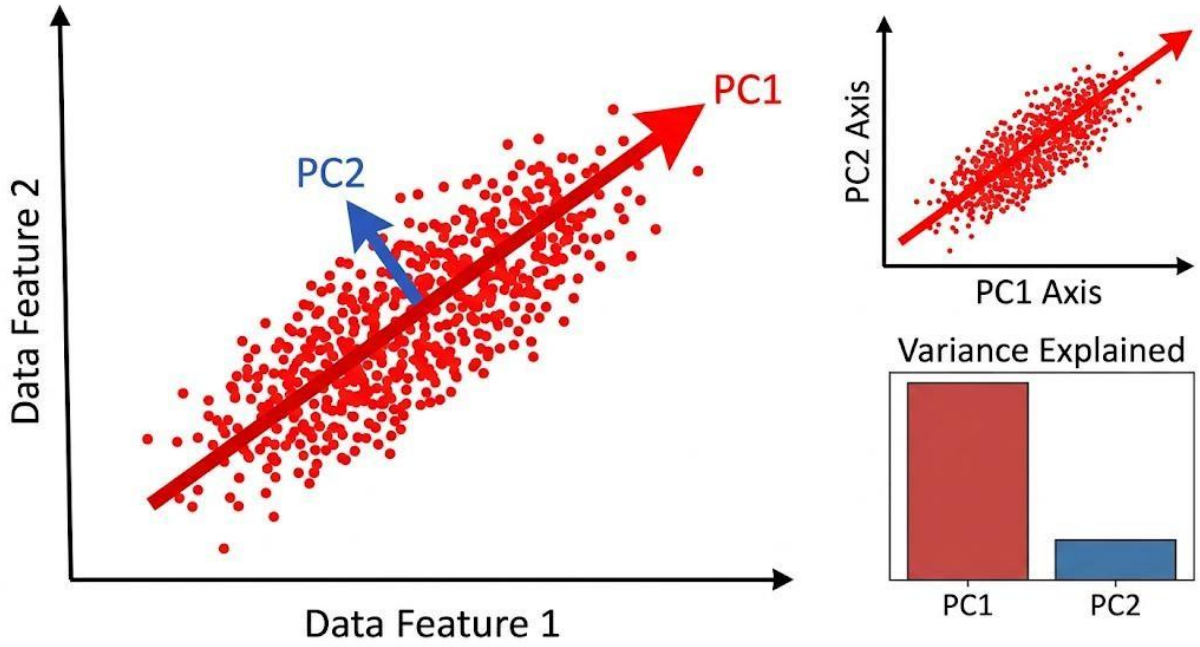
Makine öğrenmesi için kullanılan veri setlerinde binlerce tip ve yüzbinlerce farklı veri bulunur. Bunları sınıflandırmak, birbirleriyle olan ilişkilerini belirlemek ve önem katsayılarını belirlerken Özdeğer/Özvektör ilişkisi devreye girer.

Kullanıldığı Örnekler:

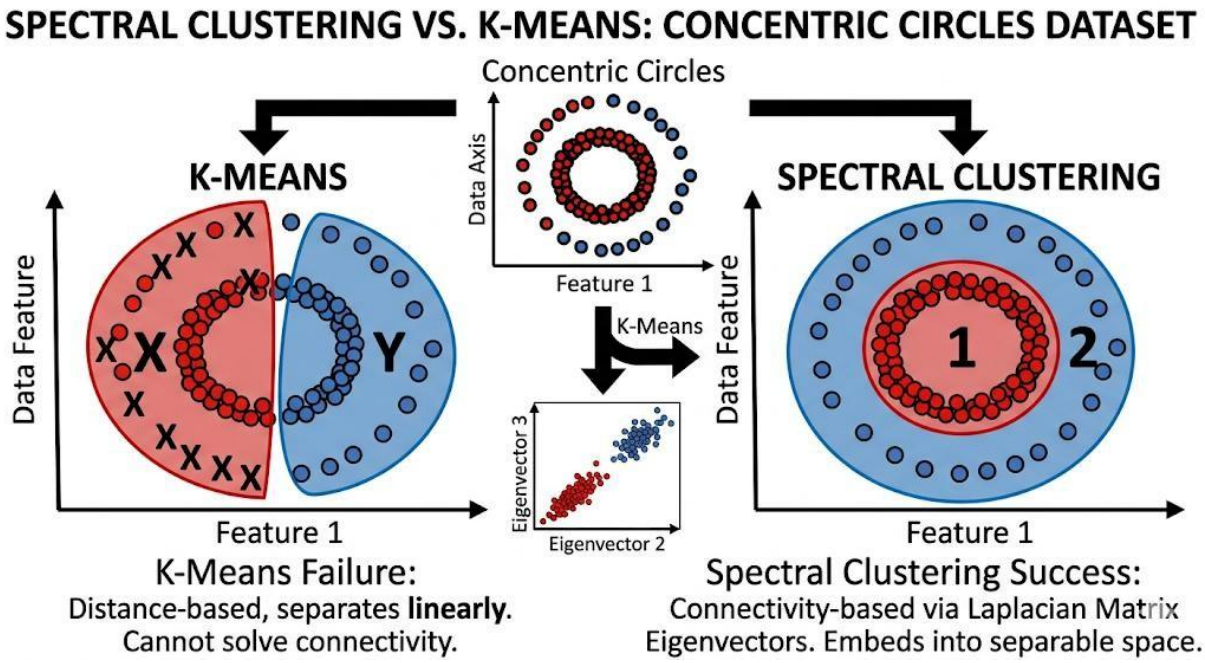
***

A. Temel Bileşenler Analizi (PCA - Boyut Azaltma): PCA'in temel amacı, veri setindeki bilginin (varyansın) çoğunu koruyarak boyut sayısını azaltmaktır.

1. Verinin özelliklerinin birbirleriyle olan ilişkisini gösteren Kovaryans Matrisi hesaplanır.
2. Bu matrisin özdeğerleri ve özvektörleri bulunur.
3. Özvektörler, verinin en çok dağıldığı (en çok varyansa sahip olduğu) yeni eksenleri (yönleri) gösterir.
4. Özdeğerler ise bu yeni eksenlerin her birinde bilginin (varyansın) ne kadarının bulunduğunu söyler. En büyük özdeğere sahip özvektör, "Birinci Temel Bileşen" olur. En küçük özdeğerli eksenler ise genelde gürültü (noise) olarak kabul edilip veri setinden atılır.

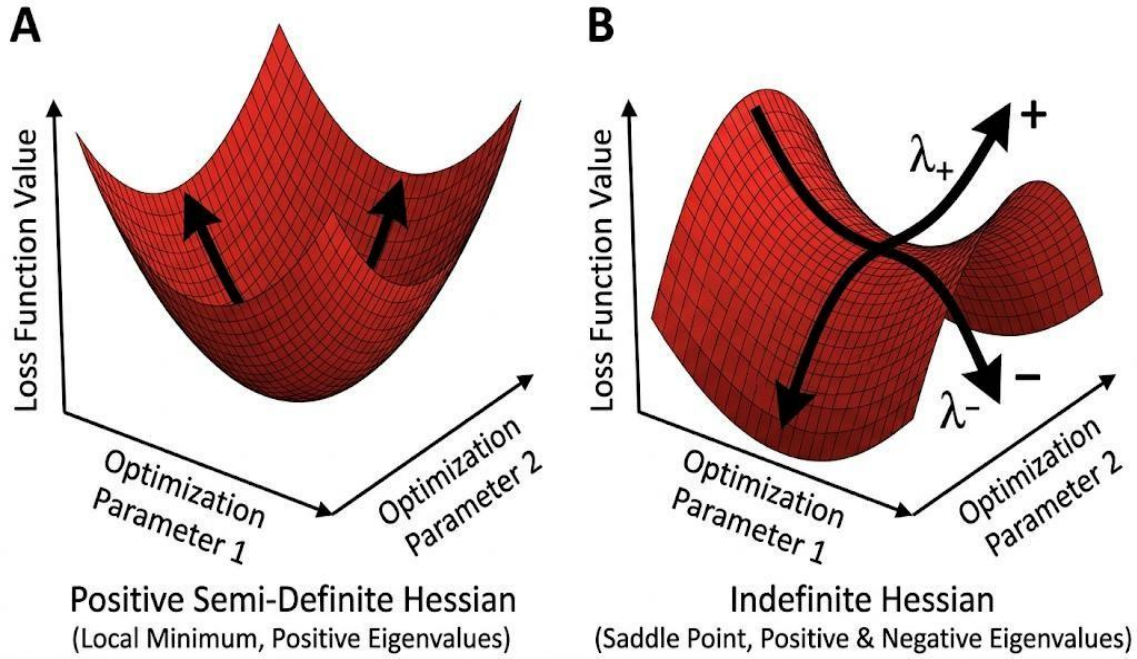


B. Spektral Kümeleme (Spectral Clustering): Geleneksel K-Means algoritması, dairesel olmayan karmaşık veri kümelerini ayırmakta zorlanır. Spektral kümeleme, veriler arasındaki benzerlikleri bir ağ (graf) yapısı olarak matrisleştirir (Laplacian Matrisi). Bu matrisin özvektörleri kullanılarak veriler çok daha düşük boyutlu ve kolay kümelenebilir bir uzaya taşınır.



C. Optimizasyon ve Derin Öğrenme: Yapay sinir ağlarını eğitirken bir "Kayıp Fonksiyonu" (Loss Function) minimize edilmeye çalışılır. Bu fonksiyonun eğriliğini gösteren matrise *Hessian Matrisi* denir. Hessian matrisinin özdeğerleri:

- Hepsi pozitifse: Doğru yerdesiniz, bir "yerel minimum" (çukur) buldunuz demektir.Hem pozitif hem negatif özdeğerler varsa: Bir "Eyer Noktası"ndasınızdır (Saddle Point), algoritma burada sıkışabilir.



***

numpy.linalg.eig Fonksiyonunun Açıklaması

Bu fonksiyon kendisine verilen kare matris için özdeğer ve özvektör hesaplamalarını yapar.

Nasıl Çalışır:

Bir kare matrisi np.linalg.eig(A) fonksiyonuna verdiğimizde arka planda sırasıyla şu işlemler gerçekleşir:
***

1. Veri Tipi ve Boyut Kontrolü (Broadcasting & Checking):

Fonksiyon ilk olarak içine gönderilen a dizisinin (array) karesel (M x M) olup olmadığını kontrol eder. Ayrıca Numpy'ın "broadcasting" kuralları geçerlidir, yani çoklu matris yığınları (stack of matrices) verilirse bunları tek tek işlemeye hazırlar.

2. Alt Seviye (Low-level) Kütüphanelere Yönlendirme (LAPACK):

Dokümantasyondaki Notes kısmında çok kritik bir cümle yer alıyor: "This is implemented using the _geev LAPACK routines".
Python, döngüler ve ağır matematiksel işlemlerde yavaş bir dildir. Bu yüzden eig fonksiyonu, işlemin kaynak kodunda (source code) doğrudan hesaplama yapmaz; matrisi alır ve doğruluğu/hızı yıllarca test edilmiş, C ve Fortran dilleriyle yazılmış LAPACK (Linear Algebra PACKage) kütüphanesinin _geev (general eigenvalue) alt programlarına iletir. Bütün ağır matematiksel çözümler bu aşamada LAPACK tarafından yapılır.

3. Özdeğerlerin ve Özvektörlerin Çözümü:

LAPACK, matrisin karakteristik denklemini çözerek özdeğerleri bulur. Özdeğerler bulunduktan sonra, denklem sistemine yerleştirilerek karşılık gelen özvektörler bulunur.
Dokümantasyon detayı: Gerçek (reel) sayılardan oluşan matrislerin karmaşık (complex) eşlenik özdeğerleri olabileceği için işlemler sırasında gerektiğinde veri tipi otomatik olarak complex (karmaşık sayı) tipine dönüştürülür (cast edilir).

4. Özvektörlerin Normalize Edilmesi:

LAPACK vektörleri bulduktan sonra, fonksiyon bu vektörleri normalize eder. Yani dokümantasyonda belirtildiği gibi "The normalized (unit 'length') eigenvectors"; vektörlerin uzunluğu 1 birim olacak şekilde ölçeklendirilir. Bu işlemler bittiğinde sonuçlar Python'a geri döndürülür.

5. Sayısal Hatalar ve İstisnalar (Round-off & Exceptions):

Kaynak kod dokümantasyonu, matematiksel yuvarlama hatalarına (round-off error) dikkat çeker. Özellikle teorik olarak birbirine tam eşit olması gereken özdeğerler (örneğin $1 + 10^{-9}$ örneğindeki gibi) bilgisayarın ondalık gösterim hassasiyeti nedeniyle farklı çıkabilir. Eğer algoritmabelli bir iterasyon sonucunda özdeğerlere yakınsayamazsa (yaklaşamazsa), fonksiyon çalışmayı durdurup bir hata fırlatır: LinAlgError.

Kod ile bulunan özdeğer ve numpy fonksiyonu ile bulunan özdeğer:

Ekte bulunan kod dosyasında, projede belirtildiği şekilde verilen kod ve fonksiyona ayrı ayrı aynı matrisi yerleştirerek sonuçları karşılaştırdık.
İkisi de değerleri başarıyla bulmakla beraber, Elle yazılan fonksiyon bir de özdeğerleri büyükten küçüğe kıyaslayıp yazdırdığından bahsetmeden geçemezdik.

***

Kullanılan Kaynaklar:

Matematiksel altyapı ve açıklamalar:

Introduction to Linear Algebra -Gilbert Strang

Matris ve vektör dönüşümlerinin ML'de yeri:

Mathematics for Machine Learning - Marc Peter Deisenroth, A. Aldo Faisal, Cheng Soon Ong.

Numpy fonksiyonu:

https://docs.scipy.org/doc/scipy/reference/generated/scipy.linalg.schur.html#scipy.linalg.schur

*(*** ile aralanan bölüm, örneklendirme yapabilmek için Gemini'dan yardım alınarak yazılmıştır***)*

Örneklemeler ve bu örneklemelerdeki kullanımın alıntılandığı kaynaklar:
Pattern Recognition and Machine Learning (PRML) — Christopher M. Bishop.
The Elements of Statistical Learning — Trevor Hastie, Robert Tibshirani, Jerome Friedman.